

Register-Resident Keccak Acceleration for ML-KEM on a RISC-V SIMT GPU

Anonymous Authors

Abstract—ML-KEM combines polynomial arithmetic with repeated Keccak permutations. On a SIMT processor, exposing parallelism within these primitives is only part of the problem: the implementation must also move their state between lanes and sustain the surrounding cryptographic workflow. We present a register-resident Keccak extension for an RV32 RISC-V GPU. Stage collectives implement column reduction, word permutation, and row logic across lane registers, while explicit low/high outputs preserve the processor’s single-destination writeback interface. A complementary NTT path uses paired butterflies and a shared bank of 16 multipliers. We integrate both paths into complete ML-KEM-768 requests and compare shuffle software, stage collectives, fused rounds, and a memory-pointer engine. On a common XRT RTL simulation build, stages accelerate complete requests by $1.462\times$ and $1.552\times$ over the same shuffle mapping at one and eight requests. Routed FPGA implementations add 1.129% LUTs over the NTT-equipped core at 250 MHz. With equal loop expansion, round fusion provides $3.291\times$ permutation throughput but only $1.033\times$ complete-request speedup in a separate timing-model control. The results show why collective granularity must be selected together with state placement and application concurrency.

Index Terms—ML-KEM, Keccak, SIMT, RISC-V, instruction-set extension

I. INTRODUCTION

The standardization of ML-KEM makes efficient post-quantum key establishment an immediate processor-design problem [1]. Its implementation alternates between modular polynomial arithmetic and SHA-3/SHAKE hashing. Both contain regular computation, but their parallelism has different shapes: an NTT layer exposes independent butterflies, whereas a Keccak round repeatedly exchanges information across a small, tightly coupled state. Accelerating one arithmetic operation therefore leaves a second question unresolved: how should a programmable processor hold and exchange that state?

An extensible SIMT GPU offers an attractive setting for this question. A warp can distribute one polynomial or one permutation over ordinary lane registers; multiple warps can process independent requests. This avoids building a separate state store for every accelerated primitive and keeps the remaining sampling, encoding, and control code on the same processor. Yet a software mapping still spends instructions on shuffles, reductions, and 64-bit operations. On RV32, each Keccak word spans two registers, making cross-word rotations and collective writeback particularly important.

There are several plausible hardware boundaries. Fine-grained arithmetic instructions leave communication in software. Stage collectives replace structured communication and logic while preserving intermediate states in GPRs. Fusing an

entire round removes additional issue and dependency costs. A pointer engine moves the state into a dedicated unit for all 24 rounds. These choices trade instruction count, pipeline storage, state transfers, and concurrency. A faster isolated permutation need not produce a faster complete key-establishment request.

Prior work supplies important building blocks. Optimized RISC-V software provides strong scalar baselines [2]; ML-Cube demonstrates a 25-thread shuffle mapping for Keccak [3]; and a published patent application describes a 64-bit warp-collective Keccak round [4]. The design question addressed here is how to realize a stage-factored interface on an RV32 SIMT datapath and determine whether coarser operations justify their implementation cost after the rest of ML-KEM is optimized.

We build this interface in Vortex [5]. Each collective consumes ordinary source vectors and produces one destination vector. Stage operations expose Keccak’s fixed communication patterns without introducing persistent accelerator state. An optimized NTT and polynomial path removes an avoidable competing bottleneck, allowing the Keccak alternatives to be compared within the same application. Our contributions are:

- A stage-factored Keccak interface with explicit RV32 low/high outputs, fixed cross-lane communication, and ordinary register dependency and writeback handling.
- A complete ML-KEM mapping that combines register-resident sponge computation with paired NTT butterflies and a shared 16-multiplier bank.
- A controlled evaluation separating software mapping, collective instructions, and round fusion, and relating their benefits to complete requests, concurrency, and routed full-core cost.

The central finding is that stages capture most of the application benefit available from register collectives. Round fusion substantially improves the permutation, but its incremental request-level gain is small; the pointer engine becomes preferable at eight concurrent requests. These outcomes make granularity a workload-dependent design choice.

II. BACKGROUND AND MOTIVATION

A. ML-KEM and its communication patterns

ML-KEM operates on degree-256 polynomials modulo 3329 and uses SHA-3/SHAKE for hashing and pseudorandom generation [1], [6]. Our workload is an ML-KEM-768 round trip: key generation, encapsulation, and decapsulation. A request in the integrated implementation executes 140 Keccak

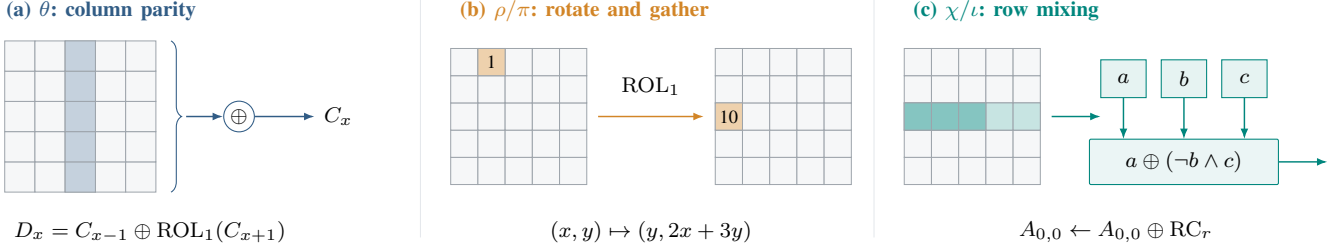


Fig. 1. Keccak’s fixed communication patterns. A state word maps to lane $x + 5y$; RV32 stores its low and high halves in two GPRs. The highlighted examples show a column reduction, the $A[1, 0]$ rotation and transfer to $B[0, 2]$, and a three-word row dependency. All coordinates wrap modulo five.

permutations, 15 forward NTTs, and nine inverse NTTs. Independent requests provide inter-warps parallelism; cooperative execution of each primitive provides intra-warps parallelism.

A Keccak-f[1600] state contains 25 words of 64 bits, arranged as $A[x, y]$ with $0 \leq x, y < 5$. Each of its 24 rounds applies column parity and correction (θ), rotation and word permutation ($\rho\pi$), and nonlinear row mixing plus a round constant ($\chi\iota$). Figure 1 illustrates these three communication patterns. Their connections are fixed by the algorithm, so a general sequence of shuffles is more flexible than the computation requires.

An ML-KEM NTT has seven layers of 128 butterflies. With coefficient $a[\ell + 32k]$ in register k of lane ℓ , the first three forward layers exchange registers within a lane. The remaining four exchange lanes at XOR distances 16, 8, 4, and 2. The inverse traverses these dependencies in reverse. This pairwise communication differs from Keccak’s reductions and permutations, motivating separate datapaths under a common register interface.

B. Why software mapping is not enough

A one-lane Keccak implementation serializes the state computation. Distributing the state across 25 lanes exposes parallel work, but the software must still form five-word parities, gather rotated words, and fetch row neighbors. In the final common-core experiment, this mapping alone reduces one-request cycles from 24.67 million for optimized RV32 assembly to 7.91 million. Hardware benefits must therefore be measured against the cooperative software, not attributed to the entire scalar-to-collective speedup.

Two constraints then shape the extension. First, an ordinary SIMT instruction writes one value per lane, while an RV32 Keccak stage produces two halves of a new state. Second, fusing more work reduces register-level communication but enlarges the operation and changes how independent warps share the execution resources. Our design addresses the first constraint with explicit result halves and evaluates the second through matched stage, round, and pointer alternatives.

Related accelerator work reinforces the importance of the data mapping. NTT-PIM and BP-NTT arrange transform operations around local storage and arithmetic capabilities [7], [8]; CHAM evaluates specialized primitives within their surrounding cryptographic computation [9]. PacQ and SynGPU simi-

larly connect SIMT data placement to datapath utilization [10], [11]. Here the state boundary is the ordinary lane register file. Compared with tightly coupled CPU extensions such as RISQ-V and scalar NTT instructions [12], [13], participating SIMT lanes supply distinct parts of one operation and receive their own results through normal vector writeback.

III. DESIGN

A. Execution model and core integration

Figure 2 shows the integration. The instruction decoder selects a collective execution unit within the ALU path. The existing operand interface supplies the source vectors; the result carries the warp and destination metadata through ordinary writeback. The scoreboard enforces register dependencies, and the unit observes the same backpressure as other execution units. Independent warps may interleave instructions without saving or restoring a private cryptographic context.

The implementation uses a 32-thread warp and 32 ALU lanes, so one issue presents the complete source vectors. Keccak assigns state to lanes 0–24 and requires all 32 lanes to execute with a full mask; padding lanes return zero. Loads and stores may be predicated, provided the warp reconverges before a collective. NTT butterflies permit pair-closed masks: each active lane’s partner must also be active. These contracts make the cross-lane operands explicit and avoid dependence on an absent or divergent producer.

B. Stage-factored Keccak

The Stage design provides θ , $\rho\pi$, and $\chi\iota$ operations. Let $\text{ROL}_{64}(v, r)$ denote a 64-bit rotation and let all state indices be modulo five. The first stage computes

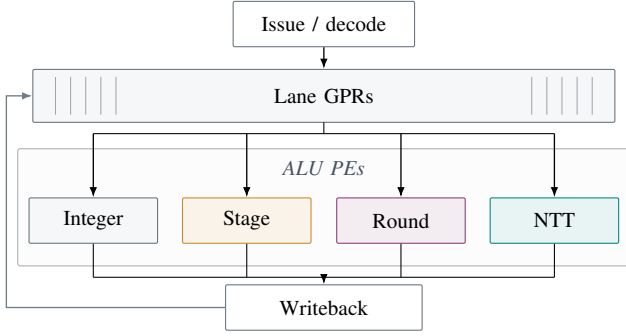
$$C[x] = \bigoplus_{y=0}^4 A[x, y], \quad (1)$$

$$T[x, y] = A[x, y] \oplus C[x - 1] \oplus \text{ROL}_{64}(C[x + 1], 1). \quad (2)$$

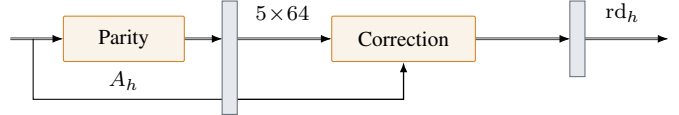
The datapath forms each parity once and distributes its correction to the five words in the column. The next stage rotates each word and routes it to a fixed destination:

$$B[y, (2x + 3y) \bmod 5] = \text{ROL}_{64}(T[x, y], r[x, y]). \quad (3)$$

(a) Register interface

(b) Stage: θ

latency / II: 2 / 1



(c) Round

latency / II: 4 / 1

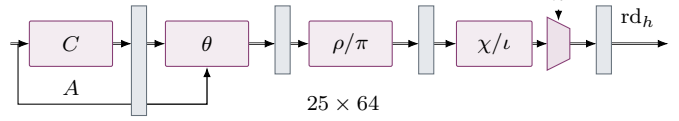
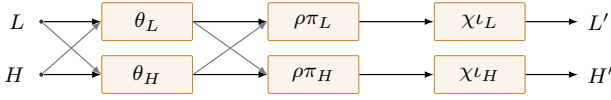


Fig. 2. Core integration. Stage, Round, and NTT units consume lane-register vectors and return one result per lane. The pointer backend instead accesses state through the LSU (not shown). Fixed collective routing is implemented in the new units. Vertical bars mark pipeline registers; h selects the output half. Latency/II refers to the unit interface, not the core pipeline.

Stage: 6 issues / round



Round: 2 issues / round

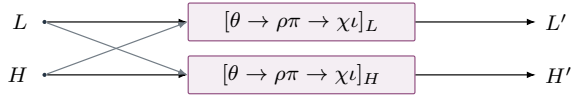


Fig. 3. Explicit RV32 result halves. Each node produces one destination vector; arrows denote data dependencies, not simultaneous issue. The θ and $\rho\pi$ low/high operations both read the same preceding pair of halves. Round fusion retains this interface while replacing the three stage pairs with one pair.

Finally, row neighbors form

$$A'[x, y] = B[x, y] \oplus (\neg B[x + 1, y] \wedge B[x + 2, y]), \quad (4)$$

with the round constant XORed into $A'[0, 0]$. These fixed connections replace repeated software reductions and gathers.

a) *Single-destination RV32 interface*: Each stage has a low-half and a high-half instruction (Fig. 3). For θ and $\rho\pi$, both instructions read the same old low/high source vectors because rotations cross the 32-bit boundary. The low result must therefore use a temporary until the high instruction has consumed the old state. In contrast, each χ^L instruction reads only its corresponding state half and a uniform round index. The interface exposes this distinction to software and lets normal RAW dependencies enforce it.

A round requires six collective instructions and a full permutation 144. Persistent state occupies two GPRs per participating lane. Temporaries add to peak register use; the looped all-stage routine uses five GPRs for state, temporaries, and loop control, with no stack frame or memory operation in the permutation loop. The θ datapath first registers the five column parities alongside the selected input half, then applies and registers the column correction. The $\rho\pi$ and χ^L paths maintain the same two-stage latency interface. All three

accept one operation per cycle when unstalled. Carrying the instruction metadata with the data preserves warp identity and destination selection through backpressure. The two-cycle unit latency excludes operand collection, issue, and core writeback delay.

C. Granularity alternatives

a) *Fused round*: Round uses four registered steps: column parity, θ correction, $\rho\pi$, and χ^L (Fig. 2). The state and instruction metadata advance together through ready/valid pipeline buffers; different warps can occupy successive steps. This supports initiation interval one while placing the intermediate 1600-bit states inside the unit. On RV32, two instructions read the same input state and select the low or high output. Each invocation computes the internal full-word round; there is no hidden result shared between the two invocations. This preserves single-vector writeback and avoids a multi-destination transaction, at the cost of recomputing internal logic for the second result half. The round index is a literal, so the implementation expands all 24 rounds. The resulting 48 collective instructions per permutation remove two-thirds of Stage's issues and its intermediate architectural states. Our evaluation also expands Stage's rounds to separate fusion from loop control.

b) *Pointer engine*: The pointer alternative accepts the address of a 200-byte state, transfers it through an LSU client, performs all 24 rounds, and writes the state back. It amortizes issue overhead over a complete permutation, but places the state outside the GPR path. This boundary changes both the surrounding sponge operations and the resources shared by concurrent requests. We compare it within the same complete application rather than treating permutation latency alone as the selection criterion.

c) *Integer width*: The RV64 version retains the same lane mapping but represents each word in one GPR. Stage then needs three instructions per round and Round needs one. This removes the explicit half selection without changing the algorithmic communication. It also widens the rest of the processor, so its benefit must be evaluated against full-core

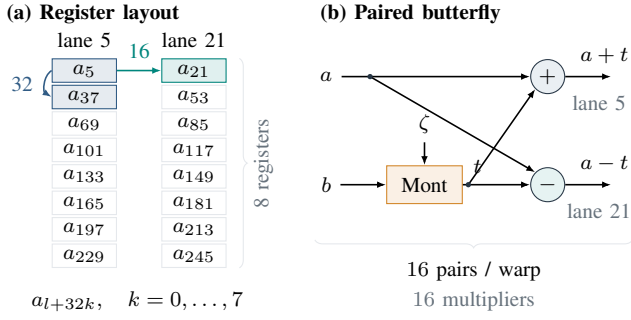


Fig. 4. NTT mapping and a forward butterfly. In the example, distance 32 connects registers within a lane, while distance 16 connects lanes 5 and 21. The pair-low lane supplies ζ ; one Montgomery product feeds both results. A 32-lane issue contains 16 such pairs.

cost and complete requests, not just the halved collective count.

D. Complementary NTT and application mapping

The polynomial path keeps 256 coefficients in eight GPRs per lane. Figure 4 shows how butterfly distance selects either another register or another lane. Lane-local operations use narrow Montgomery multiplication; cross-lane operations use paired butterfly collectives. For a forward Cooley–Tukey pair, the low lane supplies a and ζ , and the high lane supplies b :

$$t = \text{Mont}(b\zeta), \quad (a', b') = (a + t, a - t). \quad (5)$$

One product serves both lanes, and each lane receives only its own result. The inverse Gentleman–Sande form computes a reduced sum and a twiddle-scaled difference. Signed 16-bit intermediate behavior and the library’s reduction convention are preserved.

A bank of 16 signed 16×16 multipliers matches the 16 simultaneous pairs. Forward butterflies need one product phase. Inverse butterflies reuse the bank for the twiddle product and the sum-times-20159 product used by Barrett reduction. A full-warp standalone modular multiply uses the bank first for lanes 0–15 and then 16–31. Thus the bank is sized for the paired operation rather than provisioned for 32 independent products on every issue. The direct unit latency/initiation interval is 6/1 for forward butterflies and 7/2 for inverse butterflies and full-warp modular multiplication.

The integration preserves the ML-KEM library API [14]. Lane zero executes scalar control, broadcasts primitive arguments, and activates the warp for cooperative work; the wrapper then restores leader execution. One-shot SHAKE retains its state in registers across absorb, permutation, and squeeze. Incremental SHAKE stores the context between API calls. Cooperative polynomial arithmetic and a warp-level allocation arena avoid replicating data and allocations across 32 lanes. This mapping allows the Keccak backend to change while keeping NTT, polynomial work, and request semantics fixed.

TABLE I
PRIMARY EXPERIMENTAL CONFIGURATION.

Processor	RV32IM, 1 core, F/D disabled
Warp / execution width	8 resident warps, 32 lanes
Memory	16 KiB each: I-cache, D-cache, local
Outer caches	L2 and L3 disabled
Workload	ML-KEM-768 round trip, $M = 1, 8$
Integration timing	XRT RTL simulation
Physical design	Vivado 2025.1, OPT3, Alveo V80
Implementation target	250 MHz, routed core top

IV. IMPLEMENTATION AND EVALUATION

A. Experimental setup

Table I summarizes the primary configuration. We implement functional/timing models and synthesizable RTL in Vortex. XRT integration simulation executes the processor through the accelerator interface; SimX supplies additional controlled timing experiments. The headline six-backend comparison uses one all-enabled RTL build and the same application structure. Every request uses fixed FIPS-203 inputs and checks public key, secret key, ciphertext, and both shared secrets against the reference. All reported arms pass. Across these six backends, SimX and XRT agree exactly on retired instructions and differ by at most 2.71% in whole-launch cycles.

Request cycles cover the complete device launch, including entry and exit but excluding host transfers. Permutation microbenchmarks isolate the measured interval with barriers so that another warp’s initialization or output traffic cannot overlap it. Physical results are independent out-of-context core implementations, one implementation run per variant; they measure post-route cost and timing, not board execution or energy.

B. Complete-request acceleration

Figure 5 compares six Keccak backends with the final NTT and polynomial path fixed. Scalar C assigns one permutation to one lane; PQRV supplies an optimized RV32 assembly baseline. Shuffle uses the same 25-lane state placement as Stage and Round, making it the direct baseline for the collective instructions.

At one request, Stage reduces cycles from 7.91 to 5.41 million, providing $1.462\times$ speedup over Shuffle. At eight requests, the gain is $1.552\times$. Round reaches 5.20 and 12.28 million cycles, or $1.520\times/1.583\times$ over Shuffle. The larger $4.742\times/3.074\times$ gains over PQRV include the software state-distribution benefit. This separation shows that the extension improves an already cooperative implementation.

The preferred backend changes with concurrency. Round is $1.130\times$ faster than Pointer at one request, whereas Pointer is $1.374\times$ faster than Round at eight. GPR residence thus provides a useful low-request design point, while the dedicated permutation engine has the better measured batched result. The experiment establishes this crossover at the two tested request counts; it does not isolate a single arbitration or cache mechanism as its cause.

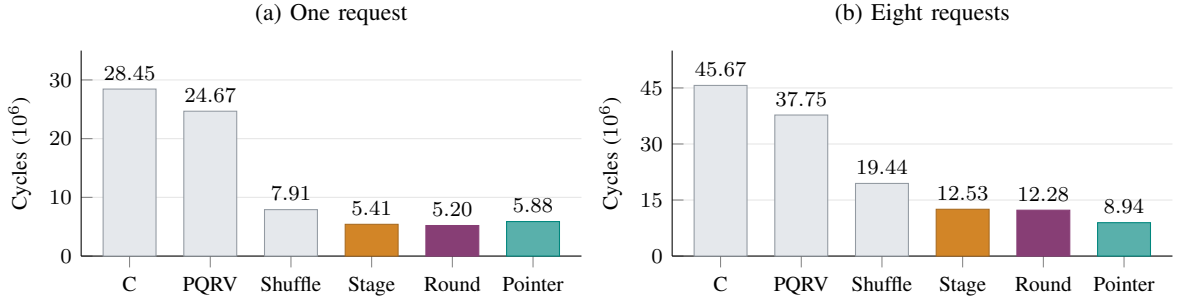


Fig. 5. Complete ML-KEM-768 round-trip cycles on one common XRT RTL configuration; lower is better. Each batch contains M independent requests. NTT and the remaining polynomial path are identical in all arms. Stage uses its looped routine; the separate equal-expansion control in Fig. 7 isolates round fusion.

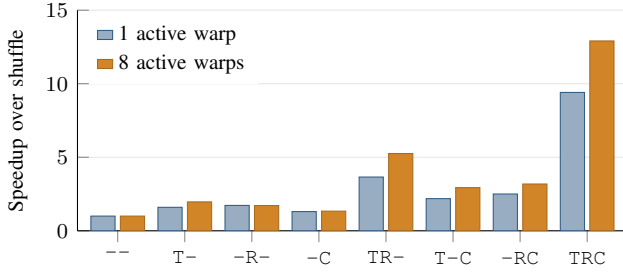


Fig. 6. Eight-way instruction-use ablation in a separate processor RTL experiment. T/R/C select θ , $\rho\pi$, and $\chi\iota$ collectives; a dash retains that software stage. Speedup uses the shuffle-only arm at the same active-warp count and the marginal cost defined in the text.

C. Which stage operations matter

Figure 6 replaces each software stage independently with its collective instruction, yielding all eight combinations. Every arm runs on the same Stage-enabled W8T32 processor RTL configuration; this separate experiment varies instruction use rather than synthesized area. To remove fixed interval overhead, it uses the marginal cost $[\text{span}(8) - \text{span}(4)]/(4W)$, where W is the number of active warps and each warp holds one state.

Using all stages improves the dependent one-warp cost by $9.408\times$ and eight-warp throughput by $12.909\times$. The best individual replacement changes with load: $\rho\pi$ gives the largest one-warp improvement, while θ gives the largest eight-warp improvement. Thus no single arithmetic replacement captures the full communication benefit. The stage interface exposes the distinct reductions and gathers, and the combined evaluation captures their interaction with the warp scheduler.

D. What additional round fusion buys

Figure 7 controls both mapping and loop expansion. Each active warp holds one state and performs 64 dependent permutations. Expanding Stage reduces branch and index overhead, but Round still achieves $3.168\times$ and $3.291\times$ the permutation throughput at one and eight active warps. The remaining benefit is therefore not an artifact of comparing an expanded round routine with a looped stage routine. These results describe the one-state-per-warp mapping; scalar software can pack

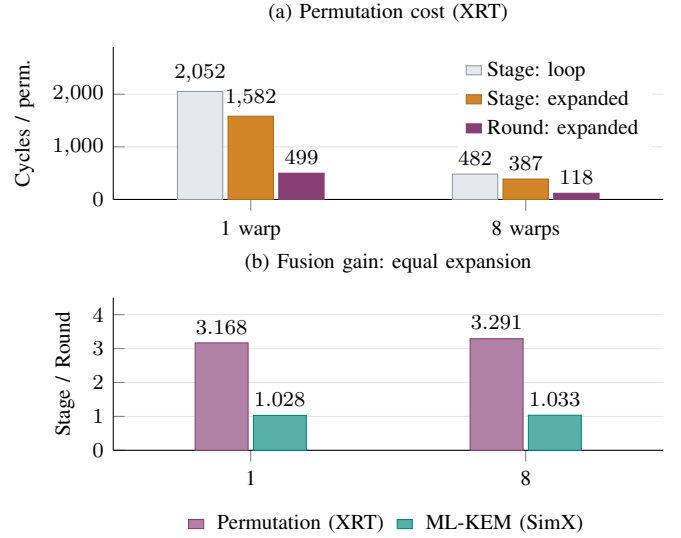


Fig. 7. Keccak granularity control in XRT: cycles per completed permutation, 64 permutations per state, one state per active warp. Panel (b) contrasts the equal-expansion gain at primitive and application scope. Permutation points use XRT; complete-KEM points use a separate matched SimX control. The x-axis denotes active warps in (a) and, in (b), active warps for permutations or concurrent requests for ML-KEM.

additional independent states and has a different occupancy tradeoff.

The same comparison is much smaller at application scope. In a separate matched SimX control with both routines expanded, Stage and Round take 5.342/5.198 million cycles at $M = 1$ and 12.336/11.938 million at $M = 8$. Round improves the complete request by only $1.028\times/1.033\times$. Removing primitive instructions is effective, but after Stage acceleration those instructions occupy too little of the complete request to deliver a proportional gain.

Figure 8 explains the saturation. Permutation accounts for 4.96% of the instrumented Stage request and 1.27% for Round, while absorb and squeeze together still occupy 11.66% and 12.90%. For Pointer, permutation is only 0.52%, but absorb and squeeze occupy 26.78%. The dominant residual shifts attention to the surrounding implementation. Matched SimX controls measure 6.6–8.3% probe overhead, so the profile sup-

TABLE II
POST-ROUTE RV32IM CORE COST AT 250 MHz. ALL VARIANTS INCLUDE
THE FINAL HALF-BANK NTT, 133 BRAM TILES, AND DISABLED F/D.

Variant	LUTs	FFs	DSPs	WNS (ns)
NTT only	220,950	131,798	112	+0.004
+ Stage	223,444	133,512	112	+0.004
+ Round	222,935	138,065	112	+0.007

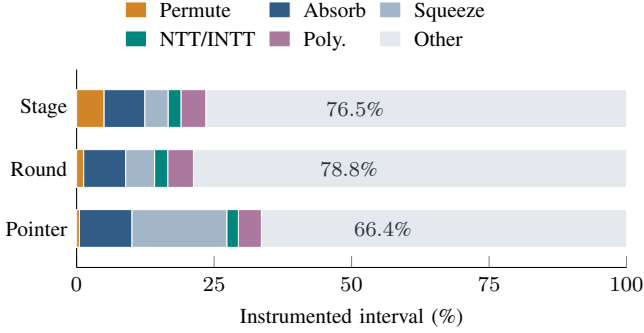


Fig. 8. Mutually exclusive XRT phase intervals at $M = 1$, using separate backend builds. Absorb and squeeze exclude nested permutations. Other includes sampling, encoding, control, memory, wrappers, and probe overhead; these instrumented intervals are not the headline launch measurements.

ports bottleneck interpretation rather than an exact prediction of uninstrumented speedup.

E. NTT communication and resource selection

The NTT design was selected against both register-shuffle and shared-memory software. Holding narrow multiplication fixed, replacing shuffle butterflies with paired collectives reduces matched SimX KEM intervals by 2.56% at one request and 8.23% at eight. The stronger shared-memory alternative groups layers and transposes coefficients; it is therefore also necessary for the final selection. In matched XRT runs, shared-memory NTT plus modular multiply takes 7.792/11.049 million interval cycles, versus 7.711/10.523 million for register collectives with the full multiplier bank. These selection runs use an earlier pointer-enabled configuration with floating-point hardware present; comparisons are confined to that family.

Reducing the bank from 32 to 16 multipliers changes complete-KEM interval cycles by at most 0.041% across matched simulation checks. Its routed implementation saves 14,767 LUTs, 1,565 flip-flops, and 16 DSPs while maintaining

250 MHz. The small application change despite an extra product phase justifies selecting the half bank. It avoids provisioning for peak lane-local multiplication when the application also depends on paired butterflies, memory, and control.

F. Routed cost and RV64 sensitivity

Table II reports independent integer-only core implementations. Relative to the NTT-equipped RV32IM core, Stage adds 1.129% LUTs and 1.300% flip-flops; Round adds 0.898% LUTs and 4.755% flip-flops. Both close 250 MHz with zero routing errors. Round uses slightly fewer LUTs but more pipeline storage, so neither is uniformly smaller across resource types. These area builds each enable one Keccak backend; the all-enabled timing build is used for comparison consistency, not for a cross-build throughput-per-area ratio.

RV64 halves the Keccak collective issue count, but complete-request instructions decrease by only 7.00% for Stage and 5.73% for Round. Matched XRT one-request cycles fall by 3.68% and 2.54%; at eight requests, Stage improves by 0.40% and Round regresses by 0.18%. The rest of the application and its memory behavior remain significant. Meanwhile, the RV64 NTT-only core uses 429,067 LUTs versus 220,950 for RV32. The RV64 Stage/Round implementations have WNS of $-0.148/-0.013$ ns at the same target, missing 250 MHz in these runs. Widening the core therefore does not provide a compelling measured tradeoff for this workload, despite simplifying the collective interface.

V. CONCLUSION

Register-resident Keccak collectives accelerate a cooperative ML-KEM implementation while retaining ordinary SIMT operands, dependencies, and writeback. Stage instructions provide substantial complete-request gains at a small incremental RV32 core cost. An optimized, compact NTT path makes this benefit visible without overprovisioning modular arithmetic. The granularity experiments identify the practical limit: whole-round fusion improves the isolated permutation much more than the complete request, and a pointer engine wins at higher tested concurrency. State placement, application phases, and concurrency must therefore be considered together when selecting a cryptographic instruction boundary.

CONTENT GENERATED BY AI

AI tools assisted with drafting, literature organization, and figure code. Experimental results come from archived implementation measurements.

REFERENCES

- [1] National Institute of Standards and Technology, “Module-lattice-based key-encapsulation mechanism standard,” NIST, Tech. Rep. FIPS 203, 2024. [Online]. Available: <https://csrc.nist.gov/pubs/fips/203/final>
- [2] J. Zhang, Y. Yan, J. Huang, and Ç. K. Koç, “Optimized software implementation of Keccak, Kyber, and Dilithium on RV{32,64}IM{B}{V},” *IACR Transactions on Cryptographic Hardware and Embedded Systems*, vol. 2025, no. 1, pp. 632–655, Dec. 2024.
- [3] T. Zhou, F. Zheng, Z. Xie, W. Tang, G. Fan, Y. Ning, Y. Bian, J. Lin, and J. Jing, “ML-Cube: Accelerating module-lattice-based cryptography using machine learning accelerators with a memory-less design,” in *Proc. ACM SIGSAC Conf. Computer and Communications Security (CCS)*, 2025, pp. 1829–1843.
- [4] M. B. Sullivan, S. Ghosh, D. R. Johnson, G. Suh, C. Joseph, and M. Y. Siu, “Warp-collective instructions to accelerate cryptographic primitives,” U.S. patent application publication US 2026/0222190 A1, Jul. 2026, applicant: NVIDIA Corp.; published July 30, 2026, Listing 3. [Online]. Available: <https://patents.justia.com/patent/20260222190>
- [5] B. Tine, K. P. Yalamarthy, F. Elsabbagh, and H. Kim, “Vortex: Extending the RISC-V ISA for GPGPU and 3d-graphics research,” in *Proc. 54th Annual IEEE/ACM Int. Symp. Microarchitecture (MICRO)*, 2021, pp. 754–766.
- [6] National Institute of Standards and Technology, “SHA-3 standard: Permutation-based hash and extendable-output functions,” NIST, Tech. Rep. FIPS 202, 2015. [Online]. Available: <https://csrc.nist.gov/pubs/fips/202/final>
- [7] J. Park, S. Lee, and J. Lee, “NTT-PIM: Row-centric architecture and mapping for efficient number-theoretic transform on PIM,” in *Proc. Design Automation Conf. (DAC)*, 2023, pp. 1–6.
- [8] J. Zhang, M. Imani, and E. Sadredini, “BP-NTT: Fast and compact in-SRAM number theoretic transform with bit-parallel modular multiplication,” in *Proc. Design Automation Conf. (DAC)*, 2023, pp. 1–6.
- [9] X. Ren, Z. Chen, Z. Gu, Y. Lu, R. Zhong, W. jie Lu, J. Zhang, Y. Zhang, H. Wu, X. Zheng, H. Liu, T. Chu, C. Hong, C. Wei, D. Niu, and Y. Xie, “CHAM: A customized homomorphic encryption accelerator for fast matrix-vector product,” in *Proc. Design Automation Conf. (DAC)*, 2023, pp. 1–6.
- [10] R. Yin, Y. Li, and P. Panda, “PacQ: A SIMT microarchitecture for efficient dataflow in hyper-asymmetric GEMMs,” in *Proc. Design Automation Conf. (DAC)*, 2025.
- [11] Y. Yao, C. Zhang, C. Qi, R. Chen, J. Wang, Z. Fu, N. Jing, X. Liang, and Z. Song, “SynGPU: Synergizing CUDA and bit-serial tensor cores for vision transformer acceleration on GPU,” in *Proc. Design Automation Conf. (DAC)*, 2025.
- [12] T. Fritzmann, G. Sigl, and J. Sepúlveda, “RISQ-V: Tightly coupled RISC-V accelerators for post-quantum cryptography,” *IACR Transactions on Cryptographic Hardware and Embedded Systems*, vol. 2020, no. 4, pp. 239–280, 2020.
- [13] P. Nannipieri, S. D. Matteo, L. Zulberti, F. Albicocchi, S. Saponara, and L. Fanucci, “A RISC-V post quantum cryptography instruction set extension for number theoretic transform to speed-up CRYSTALS algorithms,” *IEEE Access*, vol. 9, pp. 150 798–150 808, 2021.
- [14] PQ Code Package contributors, “mlkem-native,” Software repository, 2026, local dependency revision is recorded in the companion artifact. [Online]. Available: <https://github.com/pq-code-package/mlkem-native>